

MNIST 手写识别项目一

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
from torch.utils.data import DataLoader
```

```
from torchvision import datasets, transforms
```

数据加载与预处理

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])
```

准备数据集

```
train_dataset = datasets.MNIST(root='./data', train=True, download=True,
                                transform=transform)
```

```
test_dataset = datasets.MNIST(root='./data', train=False, download=True,
                                transform=transform)
```

```
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
```

```
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)
```

```
class MNISTNet(nn.Module):
```

```
    def __init__(self):
        super(MNISTNet, self).__init__()
        # 线性层: 28*28=784 个输入, 隐藏层 512
        self.linear1 = nn.Linear(28 * 28, 512)
        self.linear2 = nn.Linear(512, 256)
        self.linear3 = nn.Linear(256, 10)
        self.relu = nn.ReLU()
```

```
    def forward(self, x):
        x = x.view(-1, 28 * 28)
        x = self.relu(self.linear1(x))
        x = self.relu(self.linear2(x))
        x = self.linear3(x)
        return x
```

```
model = MNISTNet()
```

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```

```
epochs = 8
```

```
for epoch in range(epochs):
```

```

running_loss = 0.0
for batch_idx, (data, target) in enumerate(train_loader):
    optimizer.zero_grad()
    output = model(data)
    loss = criterion(output, target)
    loss.backward()
    optimizer.step()
    running_loss += loss.item()
    if batch_idx % 300 == 299:
        print(f'Epoch {epoch + 1}, Batch {batch_idx + 1}, Loss: {running_loss
/
300:.3f}')
        running_loss = 0.0
# 测试准确率
correct = 0
total = 0
with torch.no_grad():
    for data, target in test_loader:
        output = model(data)
        pred = output.argmax(dim=1, keepdim=True)
        correct += pred.eq(target.view_as(pred)).sum().item()
        total += target.size(0)
print(f'Test Accuracy: {100 * correct / total:.2f}%')

```